

# TRAVEL ANALYTICS

Comprehensive Project Explanation, Code-Verified Review, Client Q&A; and Production Readiness Guide

End-to-End Corporate Travel Analytics Pipeline

Raw Tickets → Staging → Cleansing → Enrichment → Business Rules → Analytical View → Application / BI

|                        |               |                                      |
|------------------------|---------------|--------------------------------------|
| Core requirement match | <b>82–85%</b> | Main ETL/data-engineering scope      |
| Overall PS-04 package  | <b>~74%</b>   | Weighted practical assessment        |
| Client demo readiness  | <b>~85%</b>   | Suitable for prototype demonstration |
| Production readiness   | <b>~60%</b>   | Security, BI and hardening remain    |

**Verification basis: extracted ZIP source inspection, SQLite database inspection, .pbit artifact inspection, frontend/backend source inspection, and a fresh pytest run: 18 passed with 410 warnings.**

Prepared 31 August 2026. Internal team/client-preparation document.

# 1. Executive Summary

The solution automates corporate travel-ticket analytics from vendor CSV extracts to a governed analytical dataset. Raw records are preserved in staging with batch/audit information; data is cleansed, enriched with employee and country references, interpreted with business rules, optionally overridden by an analyst, and published to a ticket-grain fact table exposed through `vw_travel`. FastAPI serves the backend API and React serves the application UI.

The project also contains value-added modules: authentication, employee portal, manager approvals, allowances, carpooling/shuttle optimization, forecasting, AI-assisted queries/support, reporting and exports. These are extensions beyond the core PS-04 requirement.

**Database reality:** the current operational database is SQLite at `backend/database/travel_analytics.db`. PostgreSQL is a recommended production migration, not the current database.

**BI reality:** `frontend/src/pages/PowerBI.jsx` is a React/Recharts BI-style dashboard using hard-coded `sampleData`. The supplied `.pbix` contains a small schema/README artifact, not a fully configured live Power BI Desktop report.

**Security reality:** login/password hashing/JWT creation exist, but most API endpoints do not enforce JWT authorization. CORS is wildcarded and secrets are hard-coded. Production hardening is required.

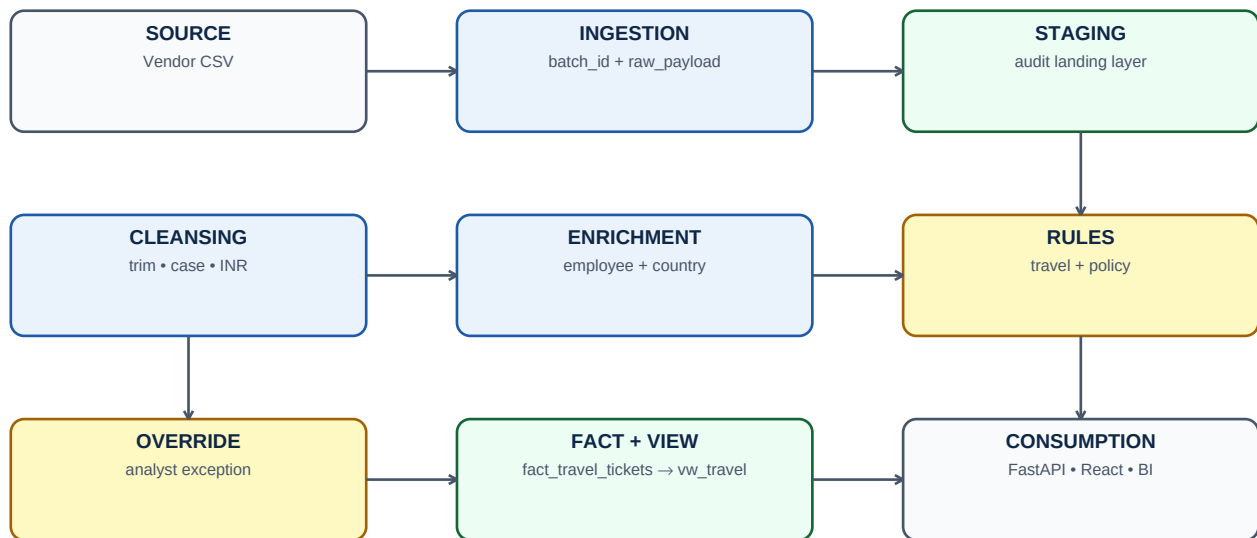
## 2. Problem Statement and Client Requirement

A multinational organization receives daily raw travel-ticket CSV extracts from a travel-management vendor. Finance and HR need rapid, consistent visibility into actual travel, booked tickets, employee context, domestic versus international movement, cross-border and multi-country travel, spend, cancellations/refunds and auditability. Manual Excel work makes the process slow and difficult to trace.

The supplied PS-04 requirement asks for an end-to-end Corporate Travel Analytics Pipeline: raw CSV → staging/audit → cleansing → employee/country enrichment → business logic → manual override → final analytical view → BI dashboard, supported by business test scenarios.

| Requirement     | Expected capability                            | Current assessment                      |
|-----------------|------------------------------------------------|-----------------------------------------|
| Staging         | Raw CSV ingestion + audit trail                | Implemented                             |
| Cleansing       | Trim, standardize, de-duplicate                | Implemented with limitations            |
| Enrichment      | Employee master + country reference            | Implemented; SCD2 temporal logic absent |
| Business rules  | Travelled flag, classification, travel_summary | Implemented                             |
| Overrides       | Documented analyst exceptions                  | Implemented                             |
| Analytical view | Governed vw_travel contract                    | Implemented                             |
| BI              | 3 required dashboard tiles                     | Partial; React/Recharts prototype       |
| Testing         | 5–7 business scenarios or equivalent           | 18 automated tests                      |
| Documentation   | Column contract/data dictionary                | Partial; formal data dictionary missing |

### 3. Architecture and Solution Flow



The architecture deliberately separates source preservation, data quality, enrichment, business interpretation, exception handling and analytical consumption. This separation makes auditability and maintenance easier.

**Lineage:** Source CSV → batch\_id → staging\_tickets/raw\_payload → cleansed\_tickets → enriched/derived record → fact\_travel\_tickets → vw\_travel → API/dashboard/BI.

## 4. Source Data and Data Model

| File                                | Rows | Role                       |
|-------------------------------------|------|----------------------------|
| backend/data/travel_raw_tickets.csv | 205  | Seeded raw ticket source   |
| Test_Dataset_Batch_1_Q3_2026.csv    | 120  | Q3 test batch              |
| Test_Dataset_Batch_2_Q4_2026.csv    | 120  | Q4 test batch              |
| backend/data/employee_master.csv    | 50   | Employee enrichment source |
| backend/data/country_reference.csv  | 10   | Country/ISO reference      |

Ticket grain is one row per ticket\_id in fact\_travel\_tickets. trip\_id groups multiple legs. Derived fields include travelled\_flag, trip\_classification, travel\_summary, policy\_compliance\_status, approval\_status and override\_applied.

The staging model stores raw fields plus raw\_payload, batch\_id and ingested\_at. The cleansed model stores normalized values and amount\_inr. EmployeeMaster stores employee context and effective dates; CountryReference stores country codes; ManualOverride stores analyst exceptions.

## 5. Database — Exact Current Answer

Current database: SQLite. SQLAlchemy constructs the connection to travel\_analytics.db. There is no operational PostgreSQL or DuckDB connection in the current implementation.

|              |              |                              |
|--------------|--------------|------------------------------|
| Staging rows | <b>5,740</b> | Raw ingestion history        |
| Cleansed     | <b>205</b>   | Unique ticket representation |
| Fact         | <b>205</b>   | Analytical ticket grain      |
| Employees    | <b>50</b>    | Current master rows          |
| Countries    | <b>10</b>    | Reference rows               |
| Batches      | <b>28</b>    | Distinct staging batches     |

| Object              | Purpose                       | Key                               |
|---------------------|-------------------------------|-----------------------------------|
| staging_tickets     | Raw/audit landing layer       | id; batch_id/ticket_id attributes |
| cleansed_tickets    | Cleansed ticket layer         | ticket_id unique                  |
| employee_master     | Employee enrichment/master    | employee_id PK                    |
| country_reference   | Country/ISO reference         | country_code PK                   |
| manual_overrides    | Analyst exceptions            | ticket_id PK                      |
| fact_travel_tickets | Final analytical ticket grain | ticket_id PK                      |
| vw_travel           | SQL analytical view           | Virtual object over fact          |
| users               | Application users             | id; email unique                  |

Why SQLite? It is lightweight and easy for a prototype. For production, PostgreSQL is the recommended target for concurrency, security, backup/operations and scale.

## 6. Code-Level Implementation Map

| File                               | Responsibility                                          |
|------------------------------------|---------------------------------------------------------|
| backend/pipeline/ingestion.py      | CSV → staging; batch ID; raw_payload                    |
| backend/pipeline/cleansing.py      | Trim/case normalization, INR conversion, duplicate flag |
| backend/pipeline/enrichment.py     | Employee + country enrichment                           |
| backend/pipeline/business_rules.py | Travelled flag, trip classification, policy rules       |
| backend/pipeline/overrides.py      | Manual override lookup                                  |
| backend/pipeline/validation.py     | End-to-end orchestration and fact publication           |
| backend/database/models.py         | SQLAlchemy models, SQLite engine, vw_travel creation    |
| backend/main.py                    | FastAPI app and endpoints                               |
| frontend/src/pages/*               | React application screens                               |
| backend/tests/test_pipeline.py     | 18 automated tests                                      |

**Batch ID:** BATCH\_YYYYMMDD\_HHMMSS\_fffff using UTC microsecond timestamp. It does not currently append a UUID.

**Raw preservation:** batch\_id + ingested\_at + raw\_payload provide auditability and traceability.

## 7. Business Rules — Exact Current Logic

| Input / condition                                                | Current result     |
|------------------------------------------------------------------|--------------------|
| ISSUED / USED / FLOWN / COMPLETED                                | travelled_flag = Y |
| CANCELLED / REFUNDED / EXCHANGED / VOID                          | travelled_flag = N |
| Unknown status                                                   | travelled_flag = N |
| Same country or IN → IN                                          | Domestic           |
| Otherwise, one origin/destination country                        | Cross-Border       |
| >1 destination country for trip_id and India not in destinations | Multi-Country      |

Policy rules currently coded: Domestic + Business cabin → NON\_COMPLIANT\_CABIN; Domestic amount > ■25,000 → NON\_COMPLIANT\_FARE; Cross-Border + Economy + amount > ■150,000 → NON\_COMPLIANT\_FARE; otherwise COMPLIANT.

**Known limitation:** the multi-country summary uses a hard-coded "{orig\_iso} to GBS Multi-Country" format. GBS is not a standard ISO country code. Generalize this before production.

## 8. Enrichment, Lifecycle and Overrides

Employee enrichment maps only employees where `is_current=1`. Missing employee IDs do not discard the ticket; fallback values are Unknown Employee, Unassigned BU and Unassigned Dept. Country names are mapped to ISO alpha-2 codes with XX for unknown values.

Manual overrides are stored separately with `ticket_id`, override travelled flag/classification/summary, reason, creator and timestamp. During fact publication, overrides are applied after derived business rules and `override_applied` is set to 1.

Cancellation/refund/exchange are treated as not travelled. Multi-leg journeys are represented by multiple ticket rows sharing the same `trip_id`; there is no dedicated `journey_leg` table.

**SCD Type-2 warning:** `employee_master` contains `effective_start_date/effective_end_date/is_current` columns, but enrichment only selects `is_current=1`. It does not yet choose the employee version based on the ticket/travel date.

## 9. Technology Stack — Briefing

| Technology         | Role                            | Why it is used                               |
|--------------------|---------------------------------|----------------------------------------------|
| Python             | Pipeline/services               | Readable ETL and data-processing ecosystem   |
| FastAPI            | REST backend                    | Typed APIs, validation, automatic docs       |
| SQLAlchemy         | ORM                             | Maps Python models to relational tables      |
| SQLite             | Current DB                      | Zero-server prototype database               |
| React 18           | Frontend                        | Reusable interactive components              |
| Vite               | Build/dev tool                  | Fast modern React tooling                    |
| Axios              | HTTP client                     | React ↔ FastAPI API calls                    |
| Recharts           | Charts                          | Interactive React analytics visuals          |
| Tailwind CSS       | Styling                         | Utility-first consistent UI                  |
| PyJWT              | JWT creation                    | Signed auth tokens; enforcement still needed |
| PBKDF2-HMAC-SHA256 | Password hashing                | Iterated password derivation                 |
| Power BI           | Intended enterprise BI consumer | Self-service Finance/leadership analytics    |

**Technology choices should be explained by responsibility: React handles application interaction; FastAPI handles service/API boundaries; SQLAlchemy handles relational persistence; SQLite is the current prototype database; BI is a separate consumption layer.**

## 10. Frontend — What the Client Sees

| Screen          | Purpose                                 |
|-----------------|-----------------------------------------|
| Login           | Authentication entry                    |
| Dashboard       | KPIs, spend, trends and classifications |
| Employees       | Directory, employee spend and allowance |
| Employee Portal | Employee self-service                   |
| Pipeline        | Run/upload/override workflow            |
| Carpooling      | Shuttle corridors, seats and savings    |
| Reports         | Executive report/export actions         |
| Forecasting     | Spend projection                        |
| PowerBI         | BI-style slicers/charts/matrix          |
| Assistant       | Natural-language analytics/support      |

React is appropriate because it supports reusable components, local state, role-based layouts and interactive dashboards. The browser should never connect directly to the database; FastAPI is the controlled service boundary.

# 11. BI / Power BI — Critical Clarification

The current PowerBI.jsx page is not an embedded live Power BI report. It imports Recharts and renders a React dashboard. Its sampleData array contains fixed business-unit spend/trip values.

Travel\_Analytics\_Dashboard.pbit is a small ZIP containing DataModelSchema, Version and README.txt. The schema describes vw\_travel and mentions SQLite/Postgres, but it is not a fully configured Power BI Desktop report with live model/report pages.

| Required BI item              | Current implementation                                    | Assessment  |
|-------------------------------|-----------------------------------------------------------|-------------|
| Trips by month                | Dashboard trend data includes hard-coded quarterly values | Partial     |
| Trips by business group       | Fact-derived BU aggregation exists                        | Implemented |
| Trips by travel-summary label | Fact-derived summary aggregation exists                   | Implemented |

**Client-safe wording:** “We have built the governed analytical layer and a BI-style application dashboard. The current demonstrator is React/Recharts; the next production step is to connect/publish a genuine Power BI report against vw\_travel.”

## 12. Testing and Verification

A fresh execution of `pytest backend/tests/test_pipeline.py -q` completed successfully: **18 tests passed** with **410 warnings**. The run completed in roughly 6.8 seconds in this environment.

The suite covers ingestion, INR conversion, duplicate processing, employee enrichment, ISO country mapping, travelled flags, domestic/cross-border/multi-country classification, manual override, `vw_travel`, forecasting, carpooling, authentication, AI assistant and executive briefing generation.

Important: passing tests do not prove production readiness. The tests do not provide load testing, browser E2E testing, security testing or statistical forecast validation. Some tests are weaker than their descriptions: for example, the duplicate test mainly checks that cleansing processes records rather than asserting a specific duplicate flag.

| Metric                 | Verified snapshot |
|------------------------|-------------------|
| Staging                | 5,740             |
| Cleansed               | 205               |
| Fact                   | 205               |
| <code>vw_travel</code> | 205               |
| Employees              | 50                |
| Users                  | 12                |

## 13. Requirement Match Scorecard

| Feature                | Status                           | Match |
|------------------------|----------------------------------|-------|
| Raw CSV ingestion      | Implemented                      | 100%  |
| Staging/audit          | Implemented                      | 100%  |
| Batch identity         | Implemented with limitation      | 90%   |
| Cleansing              | Implemented with date limitation | 80%   |
| Deduplication          | Partial                          | 70%   |
| Employee enrichment    | Partial for true SCD2            | 75%   |
| Country/ISO enrichment | Strong                           | 95%   |
| Travelled flag         | Strong                           | 95%   |
| Domestic               | Strong                           | 95%   |
| Cross-border           | Strong                           | 95%   |
| Multi-country          | Partial                          | 70%   |
| Travel summary         | Partial                          | 75%   |
| Manual overrides       | Strong                           | 100%  |
| vw_travel              | Implemented                      | 100%  |
| BI dashboard           | Partial                          | 60%   |
| Business tests         | Implemented                      | 85%   |
| Formal data dictionary | Missing                          | 0%    |

**Assessment: core ETL/data-engineering scope ≈ 82–85%. Broader PS-04 package including BI/documentation ≈ 74%. These are engineering-readiness estimates, not contractual scores.**

## 14. Additional Features Added

| Feature                   | Required?      | State / recommendation                                |
|---------------------------|----------------|-------------------------------------------------------|
| Authentication + roles    | No             | Implemented; security hardening needed                |
| Employee portal           | No             | Implemented; valuable self-service extension          |
| Manager approvals         | No             | Implemented; validate client scope                    |
| Quarterly allowance       | No             | Implemented; validate policy                          |
| Carpool/shuttle optimizer | No             | Implemented; value-added                              |
| Spend forecasting         | No             | Implemented; deterministic projection, not regression |
| AI assistant              | No             | Implemented/fallback; secure secrets                  |
| Complaint desk            | No             | In-memory prototype; persist for production           |
| OCR                       | No             | Simulation, not real OCR                              |
| C-suite briefing          | No             | HTML implemented; real PDF recommended                |
| XLSX/DOCX/PDF exports     | No             | Current Office/PDF endpoints are HTML-based           |
| Policy compliance         | No / extension | Implemented; validate policy rules                    |

**Extra-feature alignment: most extras are outside the explicit PS-04 requirement. Their business-value alignment is roughly 60–70%; they should be presented as value-added extensions, not mandatory client requirements.**

## 15. Important Technical Discrepancies

| Expectation / claim        | Actual code                                  | How to explain                                                  |
|----------------------------|----------------------------------------------|-----------------------------------------------------------------|
| UUID batch ID              | Microsecond timestamp only                   | High-resolution batch timestamp; UUID is production enhancement |
| SCD Type-2                 | Schema columns exist; temporal lookup absent | SCD2-ready structure, temporal selection still needed           |
| Date normalization         | Strings retained                             | Input is ISO-style; native DATE types should be added           |
| Live Power BI              | React/Recharts + small schema artifact       | BI-ready prototype; live Power BI publication pending           |
| Linear regression forecast | 1.12 and 1.15 multipliers                    | Deterministic trend projection                                  |
| Real OCR                   | Regex/hash simulation                        | Prototype OCR simulation                                        |
| PDF export                 | Returns HTML briefing                        | Printable HTML; true PDF generation pending                     |
| XLSX/DOCX                  | HTML with Office MIME types                  | Office-compatible prototype, not binary generation              |
| JWT security               | Token creation without route enforcement     | Authentication exists; authorization hardening needed           |
| Secrets                    | Hard-coded JWT/Gemini values                 | Move to secret manager and rotate keys                          |

# 16. Security, Reliability and Performance

| Finding                           | Severity   | Required action                                |
|-----------------------------------|------------|------------------------------------------------|
| Hard-coded Gemini API key         | Critical   | Remove/rotate; use secrets                     |
| Hard-coded JWT secret             | High       | Use secure configuration                       |
| JWT not enforced on most routes   | High       | Add auth dependency + RBAC                     |
| Wildcard CORS                     | Medium     | Restrict trusted origins                       |
| Static PBKDF2 salt                | Medium     | Use per-user random salt / modern KDF strategy |
| Per-record DB queries in pipeline | Medium     | Bulk operations / better query strategy        |
| No batch status/retry model       | Medium     | Add batch state, retries, failure alerts       |
| In-memory complaint registry      | Low/Medium | Persist support records                        |

The application is suitable for a prototype/demo, but these issues must be addressed before production deployment.

## 17. Production Target and Roadmap

| Prototype now              | Production target                                 |
|----------------------------|---------------------------------------------------|
| SQLite                     | PostgreSQL                                        |
| Timestamp batch ID         | UUID + source file/system + idempotency           |
| String dates               | Native DATE/TIMESTAMP                             |
| Simple duplicate flag      | Hash/source-aware deduplication                   |
| Current employee join      | Temporal SCD2 join                                |
| React BI prototype         | Real Power BI/Tableau enterprise report           |
| Hard-coded secrets         | Secret manager                                    |
| JWT creation               | JWT verification + RBAC                           |
| HTML exports               | True XLSX/DOCX/PDF generation                     |
| Synchronous local pipeline | Orchestrated/retryable pipeline as scale requires |
| No batch monitoring        | Batch status + metrics + alerts                   |

**Recommended priority: P0 security + real BI/export correctness; P1 SCD2/date/deduplication/database; P2 monitoring/performance/data dictionary; P3 real OCR/validated forecasting if those become approved requirements.**

## 18. Recommended Client Demonstration Flow

| Step | Action           | Message                                                  |
|------|------------------|----------------------------------------------------------|
| 1    | Problem          | Raw vendor CSVs require governed automated analysis.     |
| 2    | Upload/run       | Show batch ID and processing result.                     |
| 3    | Staging          | Show raw_payload, batch_id, ingested_at.                 |
| 4    | Cleansing        | Explain standardization and INR conversion.              |
| 5    | Enrichment       | Show employee + ISO context.                             |
| 6    | Business rules   | Show travelled/classification/summary.                   |
| 7    | Override         | Show analyst exception and reason.                       |
| 8    | vw_travel        | Explain ticket-grain governed output.                    |
| 9    | Dashboard        | Show KPIs, BU and summary visuals.                       |
| 10   | BI clarification | State React/Recharts prototype vs live Power BI.         |
| 11   | Extras           | Only then show portal, approvals, forecast, carpool, AI. |
| 12   | Close            | Explain demo readiness and production roadmap.           |

**60-second project explanation: “Our project automates corporate travel analytics from vendor CSVs. We preserve the raw source in staging with batch/audit information, cleanse and standardize it, enrich it with employee and country reference data, apply travelled and travel-classification rules, allow documented analyst overrides, and publish the result to fact\_travel\_tickets and vw\_travel. FastAPI exposes the services and React provides the application UI. We also added value-added modules for employees, approvals, allowances, forecasting, carpooling, reporting and AI support.”**

# 19. Client Question Bank — Data Engineering (1–10)

## 1. What is the source data?

**Client-ready answer:** Raw corporate travel-ticket CSV extracts from a travel-management vendor; the project includes a 205-row seed source and 120-row Q3/Q4 test batches.

**Verified evidence:** CSV files in the project root/backend/data.

## 2. What is a staging layer?

**Client-ready answer:** The first database landing area where incoming records are stored before transformation.

**Verified evidence:** staging\_tickets.

## 3. Why preserve raw data?

**Client-ready answer:** For audit, traceability and reprocessing.

**Verified evidence:** raw\_payload + batch\_id + ingested\_at.

## 4. How is batch ID generated?

**Client-ready answer:** Using UTC timestamp down to microseconds in BATCH\_YYYYMMDD\_HHMMSS\_ffffff format.

**Verified evidence:** ingestion.py; no UUID currently.

## 5. What happens when two files arrive simultaneously?

**Client-ready answer:** Microseconds reduce collisions, but formal source-file UUID/idempotency is not implemented yet.

**Verified evidence:** Production enhancement: UUID/source identity.

## 6. How are duplicates detected?

**Client-ready answer:** A seen\_ticket\_ids set marks repeated ticket IDs as is\_duplicate=1 during a cleansing run; staging history is retained.

**Verified evidence:** cleansing.py.

## 7. How is bad data handled?

**Client-ready answer:** Basic trimming/defaults/normalization exist; a formal reject/quarantine layer is still needed.

**Verified evidence:** cleansing.py; validation should be strengthened.

## 8. How are dates normalized?

**Client-ready answer:** Currently trimmed and retained as strings because supplied dates are ISO-style; native date parsing/storage is a production improvement.

**Verified evidence:** models.py uses String for date fields.

## 9. How are country codes standardized?

**Client-ready answer:** Country names are matched to the reference table and ISO alpha-2 codes; unknowns fall back to XX.

**Verified evidence:** enrichment.py + country\_reference.

## 10. How are missing employee records handled?

**Client-ready answer:** The ticket is retained with Unknown Employee, Unassigned BU and Unassigned Dept fallback values.

**Verified evidence:** enrichment.py.

## 20. Client Question Bank — Business Logic (11–20)

### 11. What is a travelled ticket?

**Client-ready answer:** ISSUED, USED, FLOWN or COMPLETED → travelled\_flag Y.

Verified evidence: business\_rules.py.

### 12. What is domestic travel?

**Client-ready answer:** Origin and destination are in the same country, including IN → IN.

Verified evidence: business\_rules.py.

### 13. What is cross-border travel?

**Client-ready answer:** Origin and destination countries differ and the trip is not multi-country.

Verified evidence: business\_rules.py.

### 14. What is multi-country travel?

**Client-ready answer:** A trip\_id has multiple destination countries and India is not in the destination set.

Verified evidence: trip\_destinations logic.

### 15. How do you handle cancellation?

**Client-ready answer:** CANCELLED → travelled\_flag N; source history remains.

Verified evidence: Status rule.

### 16. How do you handle exchange?

**Client-ready answer:** EXCHANGED → travelled\_flag N; related records may share trip\_id; no dedicated exchange chain.

Verified evidence: Status + trip\_id.

### 17. How do you handle refund?

**Client-ready answer:** REFUNDED → travelled\_flag N and excluded from travelled-spend calculations.

Verified evidence: Dashboard spend filters travelled=Y.

### 18. How do you handle multi-leg journeys?

**Client-ready answer:** Multiple tickets share trip\_id; sample demonstrates India → Germany → US.

Verified evidence: No separate journey\_leg table.

### 19. How do manual overrides work?

**Client-ready answer:** Analyst stores override values plus reason/creator; pipeline applies them after derived rules and flags override\_applied.

Verified evidence: manual\_overrides + validation.py.

### 20. How are conflicting records handled?

**Client-ready answer:** ticket\_id is the fact primary key; reprocessing updates the existing fact. A source-priority conflict engine is not implemented.

Verified evidence: validation.py.

## 21. Client Question Bank — Database (21–27)

### 21. Why this database?

**Client-ready answer:** SQLite is lightweight and serverless for the prototype; PostgreSQL is the production recommendation.

Verified evidence: models.py.

### 22. What are the major tables?

**Client-ready answer:** staging\_tickets, cleansed\_tickets, employee\_master, country\_reference, manual\_overrides, fact\_travel\_tickets, users, plus vw\_travel.

Verified evidence: models.py.

### 23. What is a view?

**Client-ready answer:** A virtual table defined by a SQL query, providing a stable interface without duplicating data.

Verified evidence: vw\_travel.

### 24. Why vw\_travel?

**Client-ready answer:** It provides a governed analytical contract so BI consumers need not understand raw pipeline internals.

Verified evidence: init\_db() creates it.

### 25. What are primary/foreign keys?

**Client-ready answer:** Primary keys uniquely identify records; foreign keys formally link tables. Current models use PK/unique constraints but do not define comprehensive ForeignKey constraints.

Verified evidence: models.py.

### 26. How are joins performed?

**Client-ready answer:** Employee and country enrichment are performed with Python maps built from master/reference tables.

Verified evidence: enrichment.py.

### 27. How do you avoid duplicate analytical records?

**Client-ready answer:** fact\_travel\_tickets.ticket\_id is the primary key; reprocessing updates an existing ticket.

Verified evidence: validation.py.

## 22. Client Question Bank — Backend (28–32)

### 28. Why FastAPI?

**Client-ready answer:** Lightweight typed REST framework with Pydantic validation and automatic API documentation.

Verified evidence: main.py.

### 29. What APIs are available?

**Client-ready answer:** Auth, dashboard, employees, tickets, approvals, pipeline run/upload/override, carpooling, forecasting, reports, PBIT, exports and AI/support.

Verified evidence: main.py.

### 30. How does React communicate with FastAPI?

**Client-ready answer:** Through HTTP /api calls using Axios; FastAPI returns JSON, HTML or file responses.

Verified evidence: frontend package + main.py.

### 31. How are errors handled?

**Client-ready answer:** Common failures use FastAPI HTTPException; centralized structured error handling/logging is not yet present.

Verified evidence: main.py.

### 32. How is validation performed?

**Client-ready answer:** Pydantic validates API payload structure; CSV validation is lighter and should add schema/type/quarantine controls.

Verified evidence: Request models in main.py.

## 23. Client Question Bank — Frontend (33–35)

### 33. Why React?

**Client-ready answer:** Reusable components, interactive state and role-based layouts fit the internal enterprise UI.

Verified evidence: App.jsx.

### 34. What information does the UI display?

**Client-ready answer:** KPIs, spend, travel classifications, employees, approvals, pipeline, reports, forecasts, carpooling, BI-style charts and AI support; employees have a portal.

Verified evidence: frontend/src/pages.

### 35. Why not directly expose the database?

**Client-ready answer:** It would expose credentials and bypass business logic/security. FastAPI is the controlled boundary.

Verified evidence: Architecture decision.

## 24. Client Question Bank — BI (36–40)

### 36. Why Power BI?

**Client-ready answer:** It is appropriate for Finance/leadership self-service reporting; the current application is BI-ready but live Power BI publication is not complete.

**Verified evidence:** PBIT schema + React BI prototype.

**Be careful:** Do not call the current React page a live embedded Power BI report.

### 37. Why not build all charts in React?

**Client-ready answer:** React is best for application workflows; BI tools are better for governed self-service analytics and enterprise reporting.

**Verified evidence:** Architecture rationale.

### 38. What KPIs are provided?

**Client-ready answer:** Total spend, total tickets, active trips, cross-border, domestic, multi-country, cancelled/refunded and pending approvals, plus BU/summary breakdowns.

**Verified evidence:** /api/dashboard/stats.

### 39. Where does Power BI get its data?

**Client-ready answer:** The intended governed source is vw\_travel; the current PowerBI.jsx uses local sampleData instead of live view data.

**Verified evidence:** PowerBI.jsx + PBIT schema.

**Be careful:** State this honestly.

### 40. How is the analytical view different from raw data?

**Client-ready answer:** It contains standardized INR, employee/ISO enrichment and derived travelled/classification/policy/approval/override fields.

**Verified evidence:** vw\_travel over fact\_travel\_tickets.

## 25. Client Question Bank — Architecture / Production (41–50)

### 41. Why separate ingestion and cleansing?

**Client-ready answer:** Ingestion preserves source truth; cleansing improves quality. Separation helps audit/reprocessing/testing.

### 42. Why separate business logic?

**Client-ready answer:** Normalization and business interpretation are different responsibilities; separation improves maintainability.

### 43. Why not one Python script?

**Client-ready answer:** A monolith becomes hard to test and change; modular stages provide clearer ownership.

### 44. Can Tableau replace Power BI?

**Client-ready answer:** Yes. Any BI tool that can consume the governed analytical source can replace the visualization layer.

### 45. Can PostgreSQL replace SQLite/DuckDB?

**Client-ready answer:** Yes. PostgreSQL is the recommended production database; current operational storage is SQLite and DuckDB is not used.

### 46. Can the pipeline process future CSV files?

**Client-ready answer:** Yes. `/api/pipeline/upload-csv` accepts a CSV and runs the pipeline; stronger schema validation/idempotency are recommended.

### 47. How scalable is the solution?

**Client-ready answer:** Architecturally extensible but prototype-scale. SQLite and per-record queries limit large volumes; PostgreSQL/bulk operations are recommended.

### 48. What happens if the pipeline fails?

**Client-ready answer:** Raw staging can preserve source history, but there is no full batch-status/retry/dead-letter framework yet.

### 49. How do you audit a result?

**Client-ready answer:** Trace dashboard → `vw_travel` → `fact_travel_tickets` → `batch_id` → `staging_tickets` → `raw_payload`.

### 50. What would you improve for production?

**Client-ready answer:** PostgreSQL, true SCD2/native dates, stronger deduplication, real Power BI/exports, JWT enforcement/RBAC, secure secrets, monitoring/retries, logging, CI/CD and deployment hardening.

## 26. Value-Added Modules — What They Really Do

**Forecasting:** the code aggregates travelled spend by business unit, then applies 1.12 for Q3 and 1.15 to Q3 for Q4. The function/docstring calls this “linear regression,” but there is no regression model. Present it as a deterministic trend projection.

**Carpooling:** four static shuttle corridors (Bengaluru, Mumbai, Hyderabad, Chennai) contain configured commuter counts, seats, costs and savings. Booking mutates the in-memory route data.

**OCR:** filename/text regex plus hash-based fallbacks simulate document extraction. It is not real OCR.

**AI assistant:** rule-based analytics/support logic with optional external AI integration. Secrets must be removed from source before deployment.

**Reports/exports:** C-suite briefing is generated as HTML. CSV is a real CSV stream; XLSX/DOCX endpoints return HTML with Office MIME types; PDF endpoint returns the HTML briefing.

## 27. Difficult Client Questions — Strong Responses

| Client challenge                                              | Recommended response                                                                                                                |
|---------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| "Why are there thousands of staging rows but only 205 facts?" | Repeated pipeline executions preserve every raw batch; fact table stays at ticket grain via ticket_id.                              |
| "Is SCD2 really implemented?"                                 | The schema is SCD2-ready, but temporal version selection is not implemented yet; this is a known gap.                               |
| "Is the Power BI dashboard live?"                             | The current demonstrator is React/Recharts; vw_travel is the governed BI source and a live Power BI report is the next deliverable. |
| "Is forecasting ML?"                                          | No. Current prototype uses deterministic growth factors; validated statistical/ML forecasting is a future enhancement.              |
| "Is OCR real?"                                                | No. Current OCR is a simulation; production requires an OCR provider.                                                               |
| "Can I trust the authentication?"                             | Authentication exists, but route-level JWT/RBAC enforcement and secret management must be hardened before production.               |
| "Why should we choose this over Excel?"                       | The pipeline makes processing repeatable, auditable and reusable, while the application provides self-service views and controls.   |

## 28. Glossary for Team Preparation

| Term           | Meaning                                                                        |
|----------------|--------------------------------------------------------------------------------|
| ETL / pipeline | Repeatable process that ingests, transforms and publishes data.                |
| Staging        | Source-oriented landing layer.                                                 |
| Audit trail    | Evidence of what arrived, when and from which processing batch.                |
| Enrichment     | Adding master/reference context.                                               |
| Business rule  | Logic that converts raw attributes into business meaning.                      |
| Fact table     | Analytical table at a defined grain; here, ticket grain.                       |
| View           | Virtual SQL table defined by a query.                                          |
| SCD Type-2     | Historical master-data technique with effective date/version tracking.         |
| Idempotency    | Reprocessing the same input without unintended duplicates.                     |
| Deduplication  | Identification of repeated logical records.                                    |
| KPI            | Key Performance Indicator.                                                     |
| RBAC           | Role-Based Access Control.                                                     |
| JWT            | Signed authentication token carrying identity/claims.                          |
| ORM            | Object-Relational Mapping; SQLAlchemy maps Python models to relational tables. |
| CORS           | Browser rule controlling allowed API origins.                                  |

## 29. Final Verdict and Team Cheat Sheet

|                |               |                                    |
|----------------|---------------|------------------------------------|
| Core ETL scope | <b>82–85%</b> | Main pipeline is strong            |
| Overall PS-04  | <b>~74%</b>   | BI/documentation/gaps reduce score |
| Frontend       | <b>~90%</b>   | Rich application prototype         |
| Production     | <b>~60%</b>   | Security/DB/BI hardening required  |
| Demo readiness | <b>~85%</b>   | Presentable as a prototype         |

**Final conclusion:** the project is a substantial, functional prototype and is suitable for a client demonstration when its current limitations are stated accurately. It should not yet be called production-ready.

### Five statements to memorize:

- “The current operational database is SQLite; PostgreSQL is the production recommendation.”
- “vw\_travel is the governed analytical contract at ticket grain.”
- “Raw data is preserved in staging with batch\_id, ingested\_at and raw\_payload for auditability.”
- “The current BI demonstrator is React/Recharts; live Power BI publication is a next step.”
- “Production hardening includes security/RBAC, SCD2, native dates, deduplication, real exports, monitoring and PostgreSQL.”

**End of comprehensive preparation guide.**